Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	Envío de mensajes distribuido
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	Uno
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Fecha	29 de septiembre de 2026 02 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo

3. Materiales y reactivos:

Diapositivas, y enunciados del problema:

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y herramientas

Computadora personal con conexión a internet

Guía de práctica.

Plantilla de matriz comparativa

5. Procedimiento / Metodología

Fase 0 (10 min). Formación de equipos de 4 (asignación del docente).

Fase 1 – Investigación (50 min). Cada integrante investiga su arquitectura y completa su sección de la matriz (estructura, ventajas, desventajas, escalabilidad, ejemplos). El equipo elabora el diagrama de su arquitectura en draw.io.

Fase 2 – Socialización (30 min). Cada equipo expone 5 min. El resto completa la matriz comparativa con la información expuesta (una sola matriz, grupal).

Fase 3 – Caso de estudio (55 min). Cada equipo selecciona la arquitectura más adecuada para el enunciado, la justifica con mínimo 3 argumentos técnicos, elabora el diagrama de la solución e identifica dónde se aplican transparencia, escalabilidad y tolerancia a fallos.

Fase 4 – Reporte y entrega (25 min). Cada estudiante redacta sus conclusiones individuales; el equipo compila la matriz final y los diagramas. Entrega en la plataforma indicada.

Criterio	Cliente-Servidor (Wagner)	Peer-to-Peer (Josue)	Multicapa (Arelys)	Microservicios (Douglas)
Estructura	La estructura está conformada por dos nodos, tanto como el nodo cliente y el nodo servidor [1]. El nodo cliente es aquel que se encarga de interactuar con el usuario, mientras tanto que el nodo servidor este se encarga de administrar la información de los clientes [1].	La estructura se basa en una red de computadoras donde los nodos también llamados peers actúan tanto como clientes y servidores.[2] Los nodos se dividen en peers simples donde los propietarios de los recursos tienen una conectividad intermitente y los super peers que cuentan con una alta capacidad y una dirección IP pública que ayuda en la localización de recursos.[2] Se puede organizar en modelos	La estructura de esta arquitectura usualmente se organiza en 3 capas [1]. La primera capa es la de presentación o usuario donde se muestra la información al usuario y se captura sus acciones o peticiones [1]. Prosigue la capa de negocio en donde están todas las reglas de dominio del sistema como algoritmos, validaciones, flujos de trabajo y procesamiento de	La aplicación se divide en servicios pequeños que trabajan juntos. Cada microservicio se encarga de una función concreta del negocio y puede ejecutarse de manera independiente. Cuando necesita información o ayuda de otro servicio, se comunica con él mediante mensajes o una API. Así, aunque cada servicio tiene su propia responsabilidad, todos colaboran para que la aplicación funcione. Por eso, es importante definir bien qué hace cada

		centralizados, no estructurados, estructurados y hibridos. [2]	datos [1]. En tercer lugar está la capa de acceso a datos que encapsula las operaciones de lectura y escritura en los medios de almacenamiento y aísla la lógica del negocio de la base de datos [1]. Finalmente se encuentra la base de datos que es el almacenamiento físico de la información de forma estructurada y persistente [1].	uno y cómo se comunican entre sí [4], [5], [6].
Ventajas	Esta arquitectura nos proporciona las siguientes ventajas: Organización de las funciones entre cliente y servidor. Centralización de la información y los recursos. Simplificación de la administración y mantenimiento del sistema. Facilita el control de acceso y la seguridad de la información.	Al ser un arquitectura descentralizada las ventajas que posee son varias tales como: una mejora en la escalabilidad al permitir a las redes peer-to-peer albergar un número ilimitados de nodos sin afectar el tiempo de respuesta de la red ; permite a los usuarios utilizar un overlay de comunicación sin repercusiones legales o de otro tipo como ataques a la intimidad manteniendo el anonimato ; la propiedad compartida la cual es una de las premisas del P2P reduce el coste de gestión de los sistemas y contenidos así como su mantenimiento [2]	Esta arquitectura ofrece varias ventajas debido a su organización jerárquica. Esto permite la separación de responsabilidades, permite el desarrollo en paralelo y facilita la mantenibilidad y modularidad de los sistemas que la implementan. Además integran un sistema de seguridad integrado al separar la capa de presentación de la base de datos. Finalmente, permite la reutilización de código desde una misma lógica de negocio hacia múltiples interfaces [1].	Se puede modificar, corregir y desplegar un servicio sin tener que desplegar nuevamente toda la aplicación. Como cada servicio se concentra en una función, resulta más fácil entender su código y darle mantenimiento. También permite que distintos equipos se encarguen de servicios específicos y trabajen con mayor autonomía. Además, si la arquitectura está diseñada para aislar los fallos, un problema en un servicio no tiene por qué detener a todos los demás [4], [5].
Desventajas	También dentro de esta arquitectura nos encontramos las siguientes desventajas: Problemas de escalabilidad cuando aumenta considerablemente el número de clientes. Mayor consumo de	Sin embargo al ser una arquitectura descentralizada las desventajas que presenta son: una alta volatilidad en los nodos que al conectarse y desconectarse de forma continua incrementa la	Por otra parte, también tiene puntos débiles. Entre ellos se destacan la alta complejidad de gestión y orquestación de los múltiples servicios e interfaces, ademas que a veces	Tener varios servicios separados también hace más complicado el funcionamiento de la aplicación. Si una operación necesita comunicarse con varios servicios, esas llamadas por red pueden causar

	recursos en el servidor debido a la cantidad de solicitudes. Dependencia del servidor, ya que una falla puede afectar a los clientes. Dependencia de la conexión de red para la comunicación entre cliente y servidor.	posibilidad de fallos en el sistema y provoca la pérdida momentánea de disponibilidad de los recursos, una sobrecarga en el ancho de banda y en los tiempos de respuesta causada por la saturación de tráfico , una mayor complejidad para garantizar la seguridad contra nodos maliciosos y para gestionar de forma eficiente recursos mutables que cambian constantemente de estado a lo largo del tiempo como los ciclos de CPU [2].	simples cambios en los datos puede requerir actualizar el esquema de la base de datos, además cada llamada puede atravesar múltiples capas lo que provoca un aumento de latencia para las consultas [1].	demoras. Cuando los datos están repartidos entre servicios, se necesitan mecanismos adicionales para mantenerlos consistentes. Encontrar un error también puede tomar más tiempo, porque hay que seguir el recorrido de la solicitud por distintos servicios. Por eso, las pruebas deben revisar tanto cada servicio por separado como la comunicación y el funcionamiento conjunto de todos ellos [4], [5], [6].
Escalabilidad	Esta arquitectura es escalable mediante el aumento de los recursos del servidor. También es posible incorporar varios servidores y distribuir las solicitudes entre ellos para atender un mayor número de clientes.	Alta capacidad de escalado global permitiendo albergar potencialmente un número ilimitado de nodos a nivel mundial. [2]	Tiene factores que favorecen la escalabilidad, como la distribución de carga de trabajo entre múltiples nodos o servidores y la eliminación de puntos únicos de fallo que permiten añadir más nodos sin degradar drásticamente el rendimiento global. Por otra parte ha implementado mecanismos modernos de escalado como los microservicios y contenedores que permiten escalar componentes según la demanda. Sin embargo, hay que considerar que a medida que crece el número de nodos, la sincronización y gestión de comunicaciones pueden verse sobresaturados [1].	Se puede aumentar la capacidad de cada servicio según la cantidad de solicitudes que recibe. Si una función tiene más demanda que las demás, se añaden más instancias del servicio encargado de atenderla, sin tener que ampliar toda la aplicación. Esto ayuda a aprovechar mejor el hardware, porque los recursos se concentran donde hacen falta. Sin embargo, cuando aumenta el número de servicios, también pueden crecer la infraestructura y el esfuerzo necesarios para administrarlos [5].
Ejemplos reales	Un sistema Universitario (EVA), ya que el estudiante	Ejemplos de uso real de esta arquitectura son en	Uno de los sistemas actuales más complejos y	Netflix y SoundCloud aparecen en los

	utiliza un navegador como cliente y el servidor de la universidad procesa las consultas.	cuanto a la compartición de archivos: BitTorrent, Kazaa, Gnutella ; plataformas académicas como DisCoP, CompuP2P y plataformas de colaboración y telefonia : Skipe y Groove. [2]	delicados que manejan multicapa son las aplicaciones bancarias. Esto debido a que manejan una capa de presentación que es la aplicación propiamente dicha con la que el usuario interactúa, pero también se accede a microservicios para validar la lógica del negocio, hacen uso de mapeadores de datos para acciones como ejecutar transacciones seguras y para la base de datos utilizan un sistema relacional como Oracle o PostgreSQL.	artículos como empresas que adoptaron microservicios [4]. SoundCloud empezó con una aplicación monolítica y tuvo dificultades de escalabilidad y mantenimiento. Para resolverlas, fue pasando funciones a nuevos microservicios conforme necesitaba modificarlas o añadir otras. En el caso de Netflix, el artículo explica que pasó de su aplicación inicial a trabajar con equipos pequeños, cada uno responsable de servicios específicos [5].

1. Cliente servidor

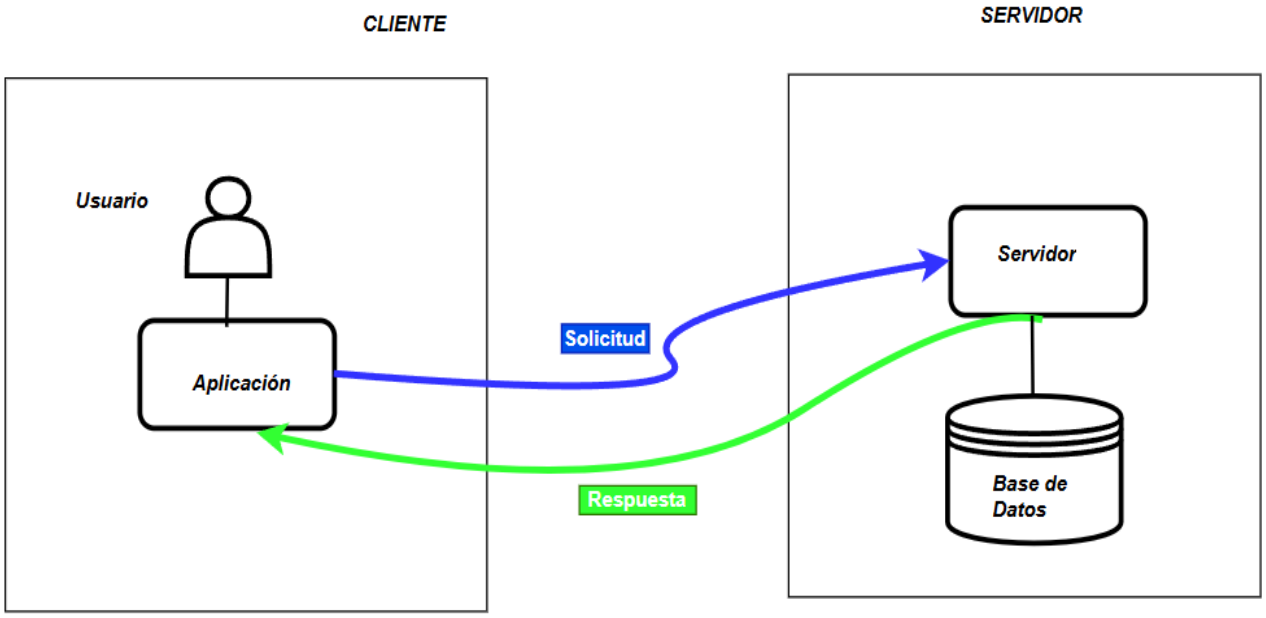


Figura 1. Ejemplo básico una arquitectura Cliente Servidor [Elaboración Propia].

2. Peer-to-Peer

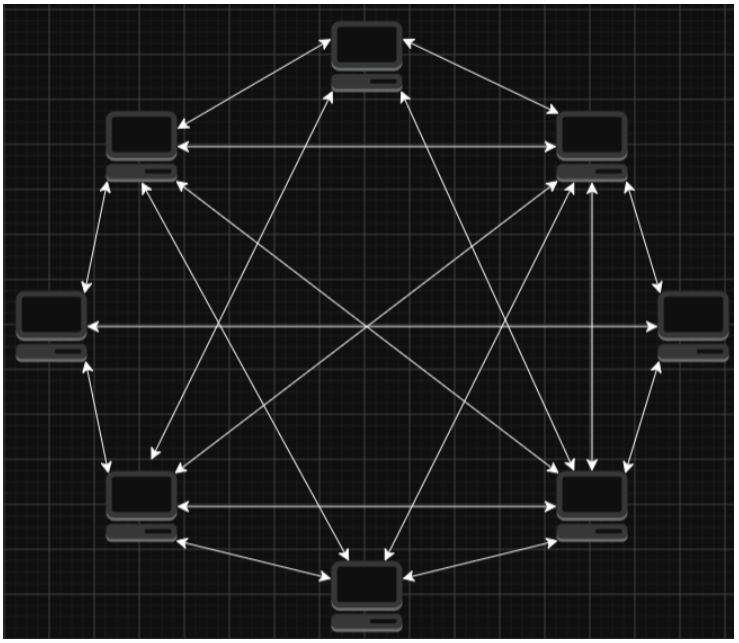


Figura 2. Ejemplo básico una arquitectura Peer-to-Peer [Elaboración Propia].

3. Multicapa

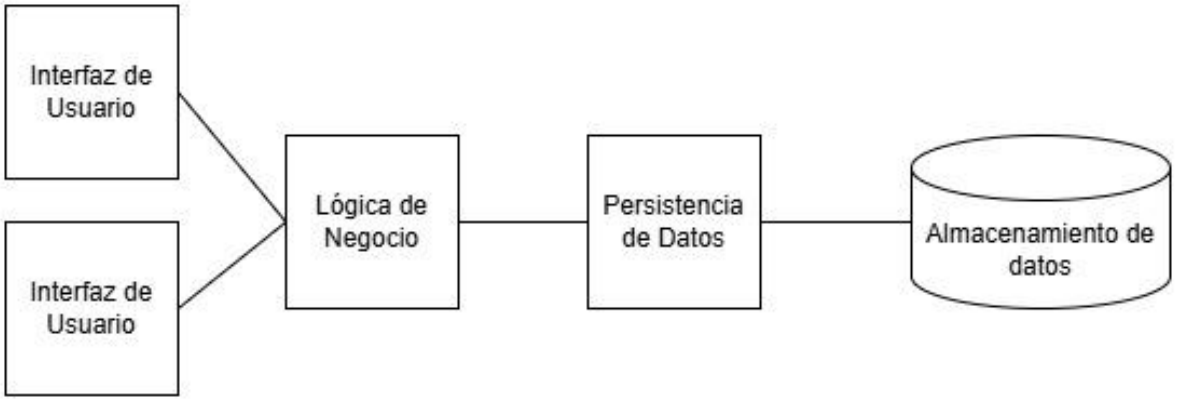


Figura 3. Ejemplo básico una arquitectura multicapa [Elaboración Propia].

4. Microservicios

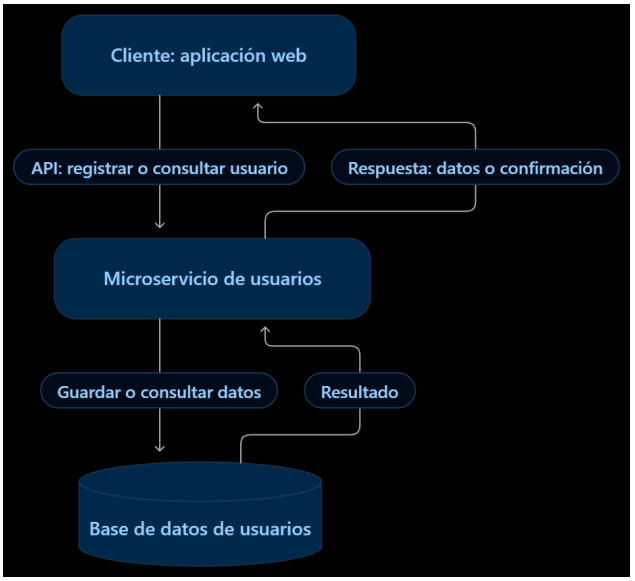


Figura 4. Ejemplo básico de un microservicio de usuarios[Elaboración Propia].

El cliente envía una solicitud al microservicio de usuarios, que se encarga de registrar y consultar usuarios. Para hacerlo, accede a su base de datos y devuelve una respuesta al cliente.

Fase 5: Socialización y matriz comparativa

1. Cada equipo expone su arquitectura durante 5 minutos (proyector)
2. Durante las exposiciones, todos los equipos llenan la matriz comparativa con la información de las cuatro arquitecturas:

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
	Estructura				

GRUPO 1 ...	Ventajas				
	Desventajas				
	Escalabilidad				
	Ejemplos reales				

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).
- Un diagrama digital de la arquitectura asignada (draw.io o similar).
- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).
- Diseño del caso de estudio: diagrama + justificación técnica (mínimo 3 argumentos) + identificación de transparencia, escalabilidad y tolerancia a fallos.
- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

7. Preguntas de Control:

- **¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?**

  Un sistema distribuido son nodos independientes conectados para funcionar como un único sistema opera mediante concurrencia entre múltiples nodos carece de un reloj global unificado, tolera fallos parciales redistribuyendo la carga de trabajo y logra una alta disponibilidad mediante el escalado horizontal, en cambio un sistema centralizado depende de un único nodo principal que procesa y almacena todas sus operaciones lo que la vuelve vulnerable a caídas totales y limita su crecimiento al escalado vertical mediante hardware más costoso

- **Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.**

  Mientras que en cliente-servidor el flujo es asimétrico y centralizado los clientes envían peticiones a un servidor central que procesa y responde obligatoriamente como intermediario lo que genera posibles cuellos de botella y puntos de fallo en cambio en peer-to-peer el flujo es simétrico directo y descentralizado ya que todos los nodos poseen el mismo rango y funcionan simultáneamente como clientes y servidores permitiendo la transferencia directa entre usuarios.

- **¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?**

  El middleware es el encargado de ofrecer conexiones persistentes entre la red o el sistema operativo y las aplicaciones que necesitan del mismo para ejecutar sus acciones. De esta manera, se puede tener varias aplicaciones o corriendo en servidores, redes o sistemas operativos diferentes, pero que funcionen como si fueran una sola. Además permite ocultar la complejidad de la infraestructura, las diferencias de hardware y los distintos lenguajes de programación.

- **En la arquitectura de tres capas (3-tier), ¿qué función cumple cada capa y por qué separarlas mejora el mantenimiento?**

  En esta arquitectura se tiene la capa de interfaz de usuario, para interactuar directamente con los clientes y recibir sus peticiones. Seguido se encuentra la capa de negocio en donde se concentran todos los algoritmos y reglas de negocio particulares del sistema. Finalmente, está la capa de persistencia de datos conformada por una base de datos que se encarga de almacenar toda la información en esquemas claros y estructurados.

- **¿Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?**

  La principal ventaja de los microservicios es que cada servicio se puede modificar, desplegar y ampliar por separado. En un diseño monolítico, un cambio puede exigir desplegar toda la aplicación; con microservicios, podemos actualizar únicamente la función que necesita ese cambio. También permiten que distintos equipos trabajen en servicios específicos y que se aumente la capacidad de los más utilizados. Además, si los fallos están bien aislados, un servicio puede fallar sin detener toda la aplicación.

  El desafío es que esas partes necesitan comunicarse por la red, lo que puede generar demoras y fallos de comunicación. También se vuelve más complicado mantener consistentes los datos repartidos entre servicios y encontrar el origen de un error. Por eso, hay que comprobar tanto que cada servicio funcione por separado como que todos trabajen correctamente juntos.

- **¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?**

  La arquitectura más adecuada sería la peer-to-peer ya que al dividir los archivos en pequeños fragmentos cada nodo que descarga partes de un archivo las comparte inmediatamente con los demás logrando que el ancho de banda total y la velocidad de la red escalen positivamente a mayor número de participantes reduciendo drásticamente los costos de servidores centrales y eliminando puntos de fallo al permitir que la descarga continúe de forma resiliente aunque el creador original se desconecte.

- **¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?**

  Influye en cómo esperamos que crezca el sistema. La escalabilidad vertical consiste en darle más capacidad a una máquina, como aumentar su memoria RAM o mejorar su procesador. Es más sencilla de aplicar, pero llega un punto en que ya no podemos seguir ampliándola. La horizontal consiste en añadir más máquinas y repartir el trabajo entre ellas. Por eso, si esperamos muchos usuarios o un crecimiento constante, conviene elegir una arquitectura que permita distribuir la carga. Eso sí, también tendremos que coordinar esas máquinas para que trabajen correctamente.

- **¿Qué significa transparencia en un sistema distribuido y mencione dos tipos de transparencia?**

  La transparencia significa que el usuario puede utilizar el sistema sin tener que conocer cómo están organizadas las máquinas que hay detrás. Aunque participen varios equipos, para él funciona como un solo sistema. Dos tipos son la transparencia de ubicación, que permite acceder a un recurso sin saber en qué máquina está guardado, y la transparencia de acceso, que permite utilizar un recurso de una forma similar tanto si está en nuestro equipo como si está en otro conectado a la red.

- **Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?**

- **En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?**

8. Evaluación

No aplica en esta practica

9. Bibliografía

w
[1] M. Lim, "C2CFTP: Direct and Indirect File Transfer Protocols Between Clients in Client-Server Architecture," in *IEEE Access*, vol. 8, pp. 104278-104290, 2020, doi: 10.1109/ACCESS.2020.2998725[1].

Josue
[2] D. Castellà Martínez, "Computación Distribuida en Entornos Peer-to-Peer con Calidad de Servicio," Tesis doctoral, Dept. d'Informàtica i Enginyeria Industrial, Universitat de Lleida, Lleida, España, mayo 2011

Arelys
[3] T. Lynn, J. G. Mooney, B. Lee, and P. T. Endo, Eds., The Cloud-to-Thing Continuum: Opportunities and Challenges in Cloud, Fog and Edge Computing, 1st ed. Cham, Switzerland: Palgrave Macmillan, 2020, doi: 10.1007/978-3-030-41110-7.

Dou
[4] N. Alshuqayran, N. Ali, and R. Evans, “A systematic mapping study in microservice architecture,” in 2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA), 2016, pp. 44–51, doi: 10.1109/SOCA.2016.15.
[5] V. Velepucha and P. Flores, “A survey on microservices architecture: Principles, patterns and migration challenges,” IEEE Access, vol. 11, pp. 88339–88358, 2023, doi: 10.1109/ACCESS.2023.3305687.
[6] M. Waseem, P. Liang, G. Márquez, and A. Di Salle, “Testing microservices architecture-based applications: A systematic mapping study,” in 2020 27th Asia-Pacific Software Engineering Conference (APSEC), 2020, pp. 119–128, doi: 10.1109/APSEC51365.2020.00020.

- Diapositivas

10. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente con FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente con FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC

11. Declaración de IA

AP